

BiREPO: Learning Goal-Conditioned Bimanual Nonprehensile Pose Reconfiguration Primitives on Diverse Objects

Anonymous Authors

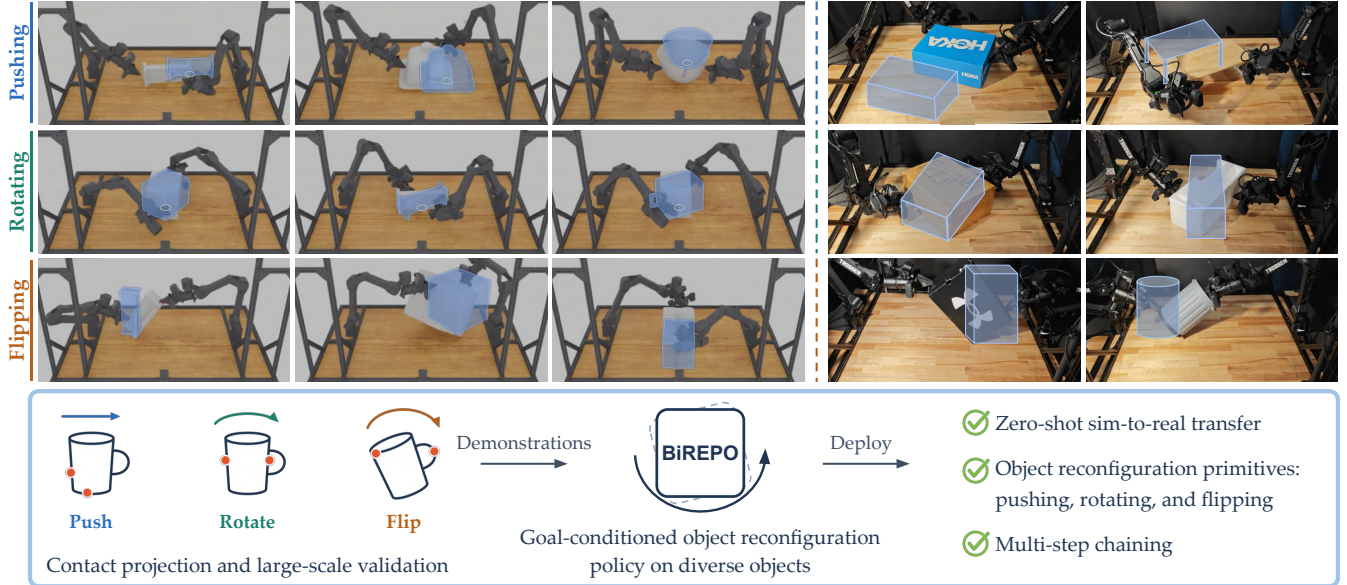


Fig. 1: BiREPO enables goal-conditioned bimanual pushing, rotating, and flipping of regular and irregular (nonconvex) objects without needing to maintain a stable grasp. We show successful example trials of BiREPO in simulation (left three columns) and on a real bimanual platform [1] (right two columns), using policies trained from simulation-generated demonstrations without human teleoperation. Above, we visualize the target object pose as a transparent overlay.

Abstract—**Bimanual nonprehensile manipulation** enables robots to move large or irregular objects that are difficult to grasp, but reliable control remains difficult: success depends jointly on coordinated contact, object dynamics, and kinematic feasibility. Learning these behaviors from teleoperated demonstrations can be costly across diverse objects and goals. We present **Bimanual REconfiguration of Poses of Objects (BiREPO)**, a framework for learning 3D bimanual nonprehensile primitives for pushing, rotating, and flipping objects without human teleoperation. We begin with geometry-based scripted primitives designed for regular shapes, then extend them to irregular (nonconvex) objects through a wrench-guided contact projection that maps nominal contact points onto arbitrary surfaces while satisfying nonprehensile objectives. We construct a data pipeline that validates candidate executions in simulation, and uses successful trajectories to train one goal-conditioned diffusion policy per primitive, conditioned on the current and goal object geometry and the requested pose change. In real-world experiments, BiREPO achieves 65%, 63%, and 70% success on pushing, rotating, and flipping, respectively. Furthermore, we demonstrate that the learned primitives can be chained together to achieve multi-step object pose reconfiguration. Supplementary material is available on our anonymous project website: <https://birepo.github.io/>.

I. INTRODUCTION

Robots operating in homes, warehouses, and manufacturing environments must reposition objects that are difficult to grasp securely because of their size or irregular geometry. Bi-

manual manipulation [2] enables coordinated contacts across different object surfaces, while nonprehensile actions [3], [4] allow robots to exploit environmental support when moving an object. Together, these capabilities enable pushing, rotating, and flipping without maintaining a stable grasp, supporting tasks such as aligning a container or turning an object onto a different face. However, reliable execution requires selecting contacts that produce the desired object motion while understanding the constraints of both arms. A contact point that works for a box can induce unintended rotation, slip, or loss of contact on an irregular-shaped object. Developing reusable bimanual behaviors therefore requires accounting for how changes in an object geometry affect the contacts needed to achieve a goal pose.

Geometry-based scripted primitives offer a practical starting point for regular shapes, but transferring their contacts to irregular objects changes the local surface normals and moment arms, potentially altering the resulting forces and torques. Our insight is to guide this transfer by the intended effect of each primitive. Pushing should generate force in the desired translation direction while limiting unwanted torque, rotation should generate the desired torque while limiting unwanted translation, and flipping should generate a moment about the supporting edge. These objectives provide a principled way to adapt nominal contacts to new geometries.

Building on these insights, we present **Bimanual RE**configuration of **P**oses of **O**bjects (BiREPO), a framework that combines wrench-guided demonstration generation with goal-conditioned policy learning for bimanual nonprehensile manipulation. BiREPO adapts contacts from regular to irregular objects using primitive-specific wrench objectives, then validates candidate executions in simulation and retains successful trajectories to train one goal-conditioned 3D diffusion policy per primitive [5], [6], generating coordinated bimanual actions from point cloud observations. The policy conditions on the current and goal object geometry, along with the requested pose change. We evaluate the resulting pushing, rotating, and flipping behaviors in simulation [7] and on a bimanual ALOHA platform [1] using regular and irregular objects, with policy comparisons and ablations. BiREPO attains the best average simulation success across our comparisons and the best pushing success, transfers zero-shot to regular and irregular real-world objects, and supports multi-step primitive chaining.

To summarize, the contributions of this paper include:

- A wrench-guided contact projection procedure that adapts bimanual manipulation primitives from regular to irregular objects by selecting contacts according to primitive-specific force and torque objectives.
- A simulation pipeline that generates, executes, and validates nonprehensile primitive demonstrations across diverse objects and goals.
- A policy architecture supporting goal-conditioned learning through cross-attention between the current and goal point clouds and an explicit goal vector, gaining 36 points over the DP3 [6] baseline.
- Validation of zero-shot deployment, evaluating BiREPO on pushing, rotating, and flipping in simulation and the real world, achieving 75% and 66% success, respectively, on both regular and irregular objects.

II. RELATED WORK

A. Bimanual Manipulation

Manipulation with two arms is critical for general-purpose robots that can flexibly perform tasks, since the arms can engage in either coordinated motions or where one arm stabilizes to help the other act [8], [9]. In either case, the movements are largely prehensile in that the robot maintains a stable grasp on the target object. Vision-Language-Action models [10] such as the π -series family [11], [12] can produce bimanual nonprehensile behavior as a result of large-scale imitation, but they require substantial proprietary real-world data. BiREPO instead learns a small set of pose-conditioned primitives entirely from simulation.

B. Robotic Data Generation in Simulation

Simulation offers a scalable way to generate abundant robot training data while reducing the need for real-world teleoperation. One paradigm is to utilize real-to-simulation-to-real [13], where simulation expands a small set of real-world demonstrations or physical interaction data to train policies for real-world deployment [14]. Related approaches

scale demonstration data through adapting trajectories [15], [16]. In contrast, our data generation pipeline begins with geometry-based scripted primitives in simulation, without requiring human demonstrations. This approach builds on the success of learning from synthetic data for grasping at scale [17], grasping with dexterous multi-fingered hands [18], [19], [20] and single-hand nonprehensile manipulation [21]. Simulation-based reinforcement learning has also enabled dual-arm grasping of large flat objects [22]. Unlike these works, we focus on *bimanual nonprehensile manipulation*, using wrench-guided contact projection to adapt primitives across object geometries and simulation to collect successful trajectories for policy learning.

C. Nonprehensile Manipulation

Classical nonprehensile manipulation includes pushing, sliding, rolling, and tilting, and has a long history [3], [4]. Planning methods often assume known object models [23], [24], or analyze contact and wrench closure in simulation [25]. Recent learning-based methods extend these ideas to 3D contact-rich control [26], [27], [28]. Some enable single-arm robots to push or tilt objects upright using parallel-jaw grippers [29], while others combine pushing and grasping of small rigid objects in single-arm planar settings [30], [31]. Bimanual approaches study push-and-grasp coordination [32] and category-level nonprehensile primitives. The closest work is BiNoMaP [33], which learns category-level bimanual nonprehensile primitives from human videos and geometric optimization. BiREPO instead learns *goal-conditioned* policies from simulation demonstrations generated by wrench-guided contact projection. Other methods learn contact-rich bimanual policies from demonstrations generated through model-based planning [34] or human demonstrations refined through optimization [35]. Our work differs in its focus on transferring bimanual contacts across object geometries while accounting for their intended force and torque effects, and studying how these primitives support goal-conditioned pushing, rotating, and flipping.

III. PROBLEM STATEMENT AND ASSUMPTIONS

We study goal-conditioned 3D bimanual nonprehensile manipulation of a single rigid object using two fixed-base robot arms with closed parallel-jaw grippers. Nonprehensile manipulation involves moving an object through coordinated external contacts and environmental support without maintaining a force-closure grasp. We consider both *regular* and *irregular* objects, where we use the term “irregular” to refer to nonconvex 3D object geometries.

We assume the object is segmented from the scene and observed through a fused point cloud P_t at each time t in the world frame. Let $T = (R, \mathbf{p}) \in SE(3)$ denote an object pose in the world (table) frame, whose $\hat{\mathbf{z}}$ axis points up, where $R \in SO(3)$ and $\mathbf{p} \in \mathbb{R}^3$. Let $T_0 = (R_0, \mathbf{p}_0)$ and $T^{\text{goal}} = (R^{\text{goal}}, \mathbf{p}^{\text{goal}})$ be the initial and desired poses. The object frame is attached at the object’s center of mass, with axes $(\hat{\mathbf{x}}_o, \hat{\mathbf{y}}_o, \hat{\mathbf{z}}_o)$, where $\hat{\mathbf{z}}_o = \hat{\mathbf{z}}$ at the initial pose. A task instance

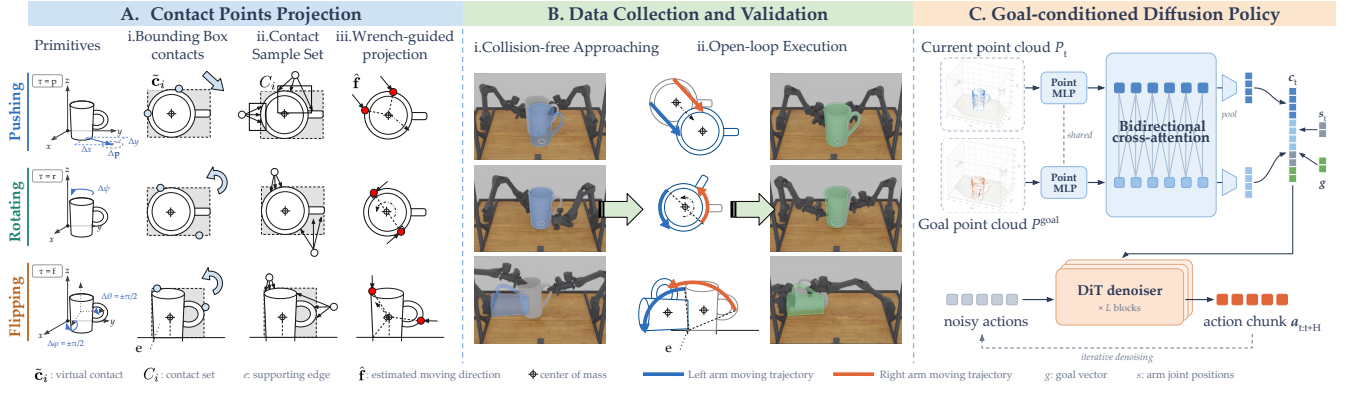


Fig. 2: Overview of BiREPO. (A, Sec. IV-A): Wrench-guided contact projection initializes bounding box contacts, samples nearby mesh contacts, and selects them using primitive-specific wrench objectives. (B, Sec. IV-B): Contacts are executed in simulation with collision-free motions and primitive-specific open-loop trajectories; successful rollouts are retained as demonstrations. (C, Sec. IV-C and IV-D): A goal-conditioned diffusion policy uses current and goal point clouds, robot states, and the goal vector to generate bimanual action chunks.

is the 4-tuple $(P_0, T_0, T^{\text{goal}}, \tau)$, where $\tau \in \{p, r, f\}$ denotes pushing, rotating, or flipping, and

$$T^{\text{goal}} = \begin{cases} (R_0, \mathbf{p}_0 + [\Delta x, \Delta y, 0]^\top), & \tau = p, \\ (R_{\hat{\mathbf{z}}(\Delta\psi)} R_0, \mathbf{p}_0), & \tau = r, \\ (R_{\hat{\mathbf{f}}} R_0, \mathbf{e} + R_{\hat{\mathbf{f}}}(\mathbf{p}_0 - \mathbf{e})), & \tau = f, \end{cases} \quad (1)$$

where $R_{\hat{\mathbf{k}}}(\alpha)$ is a rotation by α about the unit axis $\hat{\mathbf{k}}$. Pushing and rotating are defined in the world frame: a planar translation $(\Delta x, \Delta y)$, and a yaw $\Delta\psi$ about the vertical axis through the object. Flipping is defined in the object frame by a tipping direction $\mathbf{u} \in \{\pm\hat{\mathbf{x}}_o, \pm\hat{\mathbf{y}}_o\}$, which gives four possible flips, each a pitch or roll of 90° . The corresponding world-frame direction is $\hat{\mathbf{u}} = R_0 \mathbf{u}$. The object rotates by 90° about its support edge, the line where it touches the table on the side of $\mathbf{u}$, which passes through a point $\mathbf{e}$ in the world frame with direction $\hat{\ell} = \hat{\mathbf{z}} \times \hat{\mathbf{u}}$, so that $R_{\hat{\mathbf{f}}} = R_{\hat{\ell}}(\pi/2)$. We take $\mathbf{e}$ to be any point where the object touches the table that is farthest along $\hat{\mathbf{u}}$; the choice does not affect T^{goal} . The goal point cloud P^{goal} is obtained by applying the relative transform $T^{\text{goal}} T_0^{-1}$ to the initial cloud P_0 and is fixed during execution. See Fig. 2.A for illustrations.

The objective is to generate coordinated bimanual motions that move the object from T_0 toward T^{goal} . A *trial* is an execution of a task instance and is successful if the final object configuration satisfies the primitive-specific pose and settling criteria. In this paper, we use $i \in \{L, R\}$ to index the two arms, and a contact for arm i is a surface point $\mathbf{c}_i$.

IV. METHOD: BiREPO

BiREPO (Fig. 2) learns goal-conditioned policies for bimanual nonprehensile primitives across object geometries. For each object and primitive, we select the two contact points with an optimization problem (Sec. IV-A). We execute the contacts in simulation and retain successful rollouts as demonstrations (Sec. IV-B), then train one goal-conditioned 3D diffusion policy per primitive (Sec. IV-C and IV-D).

A. Contact Points Projection

We select the contact pair in two stages. We first initialize it using predefined rules on a bounding box, then project it

onto the object surface with an optimization problem.

We construct the minimum-volume bounding box containing the object with one axis vertical; its horizontal axes are $\hat{\mathbf{x}}_o$ and $\hat{\mathbf{y}}_o$. Then, we design an initial contact pair using primitive-specific geometric rules. For *pushing*, two contacts are placed behind the object relative to $(\Delta x, \Delta y)$, on different faces to oppose lateral drift. For *rotating*, we select a diagonal pair that produces the desired yaw $\Delta\psi$ and provides greater rotational leverage. For *flipping*, one arm presses downward near the top edge on the side toward which the object tips (direction $\mathbf{u}$), while the other braces the opposite side, so that the object tips about the support edge through $\mathbf{e}$. The second column in Fig. 2.A illustrates these contact selections. For all primitives, we assign the selected contact pair to the two arms based on its geometry relative to the arm bases. We denote the two virtual contacts by $\tilde{\mathbf{c}}_L$ and $\tilde{\mathbf{c}}_R$.

Because the virtual contact lies on the box, we map $\tilde{\mathbf{c}}_i$ onto the mesh to obtain realizable contacts. Projecting to the nearest surface point is not enough, since it may be poorly oriented, lie on a rim or concavity, or produce undesirable force effects. We therefore search jointly over nearby surface contacts using a primitive-specific wrench objective. A *wrench* $\mathbf{w} = [\mathbf{f}^\top, \mathbf{m}^\top]^\top \in \mathbb{R}^6$ stacks the contact force $\mathbf{f}$ and its moment $\mathbf{m}$ about a reference point. A force $\mathbf{f}_i$ at $\mathbf{c}_i$ induces the moment $\mathbf{m}_i = (\mathbf{c}_i - \mathbf{p}_0) \times \mathbf{f}_i$ about the object's center of mass $\mathbf{p}_0$; for flipping, moments are instead taken about the support edge through $\mathbf{e}$. However, the applied force depends on friction and the controller and is unavailable at planning time. Under quasi-static motion, inertia is negligible and the force primarily opposes support friction. We therefore adopt a *motion-aligned force model*, using the fingertip motion direction to approximate the applied force direction. That direction is:

$$\hat{\mathbf{f}}(\mathbf{c}) \propto \begin{cases} (\Delta x, \Delta y, 0), & \tau = p \\ \text{sign}(\Delta\psi) \hat{\mathbf{z}} \times (\mathbf{c} - \mathbf{p}_0), & \tau = r \\ r_z \hat{\mathbf{u}} - r_u \hat{\mathbf{z}}, & \tau = f \end{cases} \quad (2)$$

where $\hat{\mathbf{z}}$ is the vertical unit vector, $\mathbf{r} = (r_u, r_z) = \langle \mathbf{c} -$

$\mathbf{e}, \hat{\mathbf{u}}\rangle, c_z - e_z)$ is the contact's displacement from the support edge, used for flipping, and $\hat{\mathbf{f}}(\mathbf{c})$ is a unit vector. The corresponding moment about the task axis, vertical for pushing and rotating and the support edge for flipping, is

$$\hat{\mathbf{m}}(\mathbf{c}) = \begin{cases} [(\mathbf{c} - \mathbf{p}_0) \times \hat{\mathbf{f}}(\mathbf{c})]_z \hat{\mathbf{z}}, & \tau = p \\ \text{sign}(\Delta\psi) \|\hat{\mathbf{z}} \times (\mathbf{c} - \mathbf{p}_0)\| \hat{\mathbf{z}}, & \tau = r \\ \|\mathbf{r}\| \hat{\ell}, & \tau = f \end{cases} \quad (3)$$

Since $\hat{\mathbf{f}}$ is a unit vector, these magnitudes are distances: offset from the push line, lever arm about the vertical axis, and distance from the support edge, respectively. For each arm i , we sample the object mesh and retain the set $\mathcal{C}_i$ consisting of the 2000 samples nearest $\tilde{\mathbf{c}}_i$ whose normals face the approaching gripper and lie on a near-vertical face, and pick the pair $(\mathbf{c}_L, \mathbf{c}_R) \in \mathcal{C}_L \times \mathcal{C}_R$ using optimization:

$$\begin{aligned} \min_{\mathbf{c}_L \in \mathcal{C}_L, \mathbf{c}_R \in \mathcal{C}_R} \quad & \lambda_1 \sum_i \langle \mathbf{n}_i, \hat{\mathbf{f}}(\mathbf{c}_i) \rangle + \lambda_2 s_F \left\| \sum_i \hat{\mathbf{f}}(\mathbf{c}_i) \right\| \\ & + \lambda_3 s_M \left\| \sum_i \hat{\mathbf{m}}(\mathbf{c}_i) \right\| + \lambda_4 \sum_i \|\mathbf{c}_i - \tilde{\mathbf{c}}_i\| \\ \text{s.t.} \quad & \text{dist}(\mathbf{c}_i, \partial\mathcal{S}) \geq \rho, \quad i \in \{L, R\}, \end{aligned} \quad (4)$$

with $s_F = 1$ for rotating and 0 otherwise, and $s_M = +1$ for pushing and -1 for rotating and flipping. Here $\mathbf{n}_i$ is the outward surface normal at $\mathbf{c}_i$, $\mathcal{S}$ the object surface, $\partial\mathcal{S}$ its edges, ρ a predefined threshold, and $\{\lambda_1, \dots, \lambda_4\}$ are hyperparameters. Intuitively, the first term favors contacts pushed into instead of skimmed across, reducing slip. For rotating, the second favors opposing fingertip motions, reducing net translation. The third minimizes net moment for pushing, while maximizing it for rotating and flipping. The final term keeps contacts near their box candidates, while the constraint avoids edges, rims, and handle openings.

B. Data Generation in Simulation

1) *Task setup*: We generate demonstrations in Isaac Lab [7], using a simulated dual-arm setup matching our real system [1]. We use 100 object meshes from [36]. For each mesh, we identify the orientations in which the object rests stably as candidate starting orientations. For flipping, we retain only object-direction pairs (object, $\mathbf{u}$) whose configuration (after a 90° flip in direction $\mathbf{u}$) remains stable in a robot-free simulation. In each episode, objects spawn with ± 5 cm position jitter and uniformly sampled yaw. Push goals translate the object by 0.20 m in a uniformly sampled planar direction, rotation goals sample $\Delta\psi \sim \mathcal{U}[-90^\circ, 90^\circ]$, and flip goals specify a 90° flip in direction $\mathbf{u}$.

2) *Motion generation*: Given the projected contacts $\mathbf{c}_i$ on the object at its initial pose T_0 , we obtain the goal position $\mathbf{c}_i^{\text{goal}}$ of each contact by applying the relative transform $T_0^{\text{goal}} T_0^{-1}$ to $\mathbf{c}_i$. Each episode then proceeds in three phases. Before contact, both arms move to their contacts $\mathbf{c}_i$ along collision-free trajectories planned with cuRobo [37]. In contact, each fingertip follows the path that its contact point traces as the object moves from T_0 to T^{goal} ,

$$\mathbf{c}_i(s) = \begin{cases} \mathbf{c}_i + s [\Delta x, \Delta y, 0]^\top, & \tau = p, \\ \mathbf{p}_0 + R_{\hat{\mathbf{z}}}(s \Delta\psi) (\mathbf{c}_i - \mathbf{p}_0), & \tau = r, \\ \mathbf{e} + R_{\hat{\ell}}(s \pi/2) (\mathbf{c}_i - \mathbf{e}), & \tau = f, \end{cases} \quad (5)$$

which, for $s \in [0, 1]$, runs from $\mathbf{c}_i$ at $s = 0$ to $\mathbf{c}_i^{\text{goal}}$ at $s = 1$. For pushing, this is the straight line between the two points; for rotating, an arc about the vertical axis through the center of mass, with radius equal to the horizontal distance from $\mathbf{c}_i$ to that axis; and for flipping, an arc about the support edge, with radius equal to the distance from $\mathbf{c}_i$ to the edge. After contact, both arms return to their rest poses along collision-free trajectories, again planned with cuRobo.

3) *Data filtering*: As geometrically feasible contacts can still fail due to contact dynamics, we retain only successful rollouts. Pushing and rotating trials require an xy position error below 30 mm and an orientation error below 10° relative to T^{goal} . For flipping, the object must rotate to within 25° of the desired 90° rotation, and remain on the resulting resting face after the arms withdraw. We evaluate each rollout from the initial contact approach to the final resting pose. Successful episodes are recorded at 10 Hz, and contain the object point cloud, arm joint positions, goal specification, and commanded joint targets used for policy learning (Sections IV-C and IV-D). The object point cloud P_t is reconstructed from two simulated third-person cameras matched to our real setup, cropped to the object, and corrupted with sparse Gaussian noise to mimic outliers [34].

C. Policy Architecture

We train one goal-conditioned 3D diffusion policy [5], [6] per primitive on the demonstrations from Sec. IV-B. At time t , an observation consists of the object point cloud P_t and arm joint positions $\mathbf{s}_t$. The policy takes the $H_o = 2$ most recent observations, together with P^{goal} and a goal vector $\mathbf{g}$, as input. It outputs a chunk [1] of H_p absolute joint position targets $\mathbf{a}_{t:t+H_p}$, of which the first H_a are executed before replanning. We set $H_p = 64$ and $H_a = 32$.

1) *Goal-conditioned cross-attention*: Independently encoding and pooling P_t and P^{goal} would prevent point-level interactions between the current and desired object surfaces. Instead, we use a shared point encoder followed by two layers of bidirectional cross-attention between the current and goal point clouds before max pooling (see Fig. 2.C).

2) *Goal vector*: In addition to the goal point clouds, we give the policy the commanded pose change explicitly. From the initial and goal poses of Sec. III,

$$\begin{aligned} \Delta R &= R_0^{\text{goal}} R_0^\top, & \Delta \mathbf{t} &= \mathbf{p}^{\text{goal}} - \mathbf{p}_0, \\ \mathbf{g} &= [\Delta \mathbf{t}^\top, \text{r6d}(\Delta R)^\top]^\top \in \mathbb{R}^9, \end{aligned} \quad (6)$$

where $\text{r6d}(\cdot)$ stacks the first two columns of a rotation matrix, a continuous rotation representation [38]. Both terms are expressed in the world frame, and P_0^{goal} is P_0 transformed by $(\Delta R, \Delta \mathbf{t})$ about $\mathbf{p}_0$. For pushing, $\mathbf{g}$ encodes a planar translation; for rotating, a yaw; and for flipping, the resulting rotation and center-of-mass displacement.

Primitive	Random	Normal	Wrench-guided
<i>Attempts per successful demo (APS) ↓</i>			
Pushing	5.4	4.1	2.1
Rotating	5.9	4.8	1.9
Flipping	3.2	1.6	1.6
<i>Objects with fewer than 5 demos / % ↓</i>			
Pushing	40	30	9
Rotating	19	24	4
Flipping	16	6	6

TABLE I: Comparison of random, normal, and wrench-guided contact projection strategies over 100 objects. We report attempts per successful demonstration and the percentage of objects with fewer than five demonstrations (lower is better for both).

3) *Conditional action generation*: The resulting geometry features are combined with the corresponding joint positions and the goal vector $\mathbf{g}$ to form a conditioning vector $\mathbf{z}_t$. A Diffusion Transformer [39] denoiser $\varepsilon_\theta(\mathbf{a}_{t:t+H_p}^k, k, \mathbf{z}_t)$ treats each action in the chunk as a token and generates the chunk by iterative denoising (see Fig. 2.C and Sec. IV-D).

D. Policy Training and Inference

We train each policy using the DDPM noise prediction objective [40] with $K = 100$ diffusion time steps and a squared-cosine schedule [41]. For a demonstrated action chunk $\mathbf{a}_{t:t+H_p}$, we sample $k \sim \mathcal{U}\{1, \dots, K\}$ and Gaussian noise $\varepsilon \sim \mathcal{N}(0, I)$. Let $\mathbf{a}_{t:t+H_p}^k$ denote the corresponding noisy action chunk obtained from the standard DDPM forward process [40]. We jointly train the encoder and denoiser end-to-end from scratch by minimizing

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathcal{D}, k, \varepsilon} [\|\varepsilon - \varepsilon_\theta(\mathbf{a}_{t:t+H_p}^k, k, \mathbf{z}_t)\|_2^2], \quad (7)$$

where the expectation is over simulation-generated demonstration windows sampled from the dataset $\mathcal{D}$, diffusion time steps k , and Gaussian noise ε . At inference time, we compute $\mathbf{z}_t$ once per replanning step and generate an action chunk using 10 DDIM steps [42]. At 10 Hz each plan covers 6.4 s, of which 3.2 s are executed before re-observing.

V. EXPERIMENTS

We evaluate BiREPO by investigating: (i) How much does wrench-guided contact point projection improve demonstration generation on irregular objects? (ii) How does BiREPO compare with alternative policy architectures and ablations in simulation? (iii) How well do the learned policies transfer zero-shot to real-world objects and compare with prior bimanual nonprehensile methods? (iv) Can the learned primitives be chained for multi-step pose reconfiguration?

A. Contact Projection Results

Table I compares three approaches for mapping virtual contacts to object-surface contacts: random sampling from the candidate contact pool, a normal projection that casts a ray along the proxy-face normal and selects the first intersection, and our wrench-guided projection (Sec. IV-A). For each of the 100 objects [36] and each projection method, we attempt 50 demonstrations in simulation. We

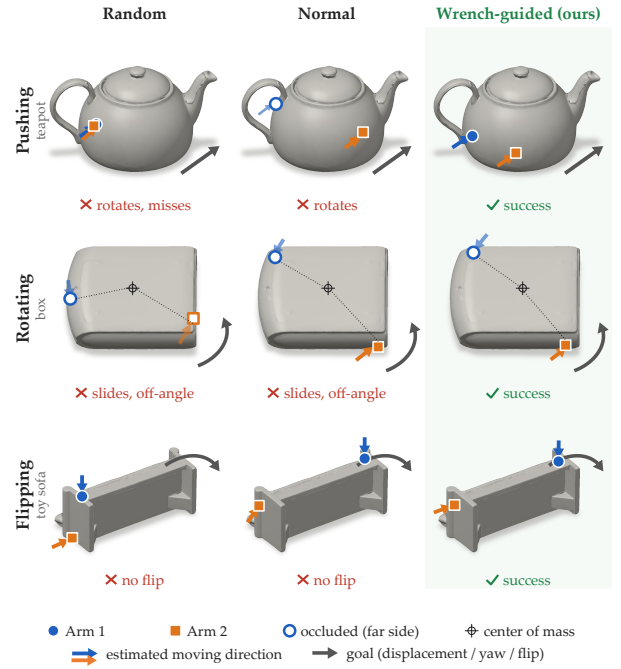


Fig. 3: Qualitative moving comparison of random, normal, and wrench-guided (ours) contact projection for the three primitives. Our projection selects contacts that better match the desired primitive motion, avoiding unintended rotation, translation, or failed flipping. Arrows indicate applied contact forces and desired object motion.

report *attempts per successful demonstration (APS)*, the average number of rollout attempts required for one successful demonstration and the percentage of objects yielding fewer than five demonstrations.

Random sampling is the least efficient method, requiring 4.8 APS on average across the three primitives. Wrench guidance provides the largest gains for pushing and rotating: compared with normal projection, APS decreases from 4.1 to 2.1 for pushing and from 4.8 to 1.9 for rotating. It also improves coverage, reducing the percentage of objects with fewer than five demonstrations from 30% to 9% for pushing and from 24% to 4% for rotating. For flipping, wrench-guided and normal projection perform similarly.

Fig. 3 illustrates why wrench guidance helps. Normal projection can support the desired motion but also induce unwanted motion. For example, the teapot rotates during pushing and the box slides during rotation. Our wrench-guided objective explicitly penalizes these undesired components. The toy sofa example also highlights the importance of avoiding object edges, where normal projection produces a contact that fails to complete the flip.

B. Simulation Architecture Ablations

We evaluate our architecture through ablations on cross-attention and the goal vector, and compare against a variant of BiREPO and a standard point cloud-based baseline:

- 1) **BiREPO-ACT**: uses the same point cloud encoders, goal inputs, cross-attention, and action chunking as in BiREPO, but replaces the diffusion transformer with the encoder-decoder from ACT [1] and the diffusion objective with the CVAE [43] objective.

Primitive	Criterion	BiREPO (DiT) (Ours)	BiREPO-ACT	Ours w/o Pose Vector Input	Ours w/o Cross Attention	Ours w/o Either	DP3
Pushing	Pose ERR (mm) ↓	13.02 ± 0.31	18.50 ± 0.17	11.90 ± 0.39	16.48 ± 0.22	14.40 ± 0.63	67.19 ± 2.03
	Orientation ERR (deg) ↓	3.69 ± 0.06	5.49 ± 0.21	4.23 ± 0.13	4.11 ± 0.02	3.96 ± 0.23	7.61 ± 0.40
	Success (%) ↑	76.54 ± 0.39	62.72 ± 0.84	74.81 ± 1.58	69.20 ± 1.39	71.23 ± 1.54	17.96 ± 0.49
Rotating	Pose ERR (mm) ↓	8.54 ± 0.17	10.94 ± 0.33	8.60 ± 0.36	9.10 ± 0.83	8.73 ± 1.38	13.13 ± 0.43
	Orientation ERR (deg) ↓	4.99 ± 1.07	3.95 ± 0.41	6.98 ± 0.99	5.99 ± 1.45	7.09 ± 0.79	13.67 ± 2.42
	Success (%) ↑	70.63 ± 7.54	76.39 ± 4.62	59.92 ± 4.99	64.09 ± 2.09	59.52 ± 0.60	40.48 ± 2.38
Flipping	Orientation SR (%) ↑	78.38 ± 1.33	80.99 ± 0.71	80.44 ± 2.81	70.66 ± 3.78	75.35 ± 1.45	58.54 ± 0.63
	Settled (%) ↑	83.33 ± 2.28	84.99 ± 0.63	84.43 ± 1.67	77.27 ± 2.52	80.58 ± 1.49	74.93 ± 0.48
	Success (%) ↑	77.69 ± 1.49	80.85 ± 0.95	79.89 ± 3.04	70.38 ± 3.75	74.24 ± 1.67	57.58 ± 1.26
Average	Success (%) ↑	74.95 ± 2.43	73.32 ± 1.06	71.54 ± 1.33	67.89 ± 1.57	68.33 ± 0.95	38.67 ± 1.28

TABLE II: Simulation experiment results across all architecture ablations and baselines, reported as mean $\pm$ standard deviation over three seeds. For pushing and rotating, success requires the object’s final position and orientation errors to be within 30 mm and 10° of the goal, respectively. For flipping, success requires satisfying both the orientation and settling criteria (see Sec. IV-B). Pose ERR and Orientation ERR are computed for each seed as the median over all evaluation rollouts, including failures, and then aggregated across seeds. We use the median to reduce sensitivity to a small number of large errors. Bold marks the best entry in each row.

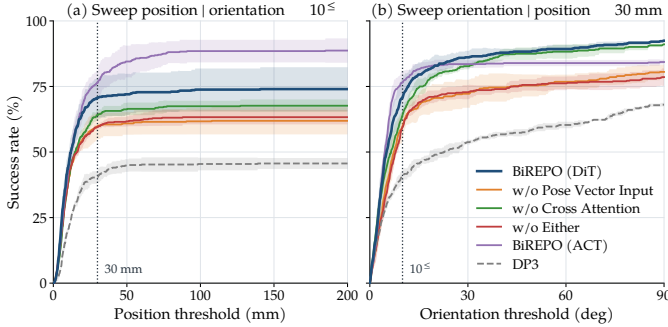


Fig. 4: Sensitivity of simulation success to the pose error thresholds for rotating 10° (left), and the orientation threshold with position fixed at 30 mm (right). The dashed vertical lines mark the thresholds used in Table II for quantifying success.

- 2) **DP3**: we adapt standard DP3 [6] to our goal-conditioned setting by concatenating the current and goal point clouds with an indicator distinguishing goal points, while otherwise retaining the original DP3 architecture and U-Net [44] backbone. Unlike BiREPO, this baseline does not use cross-attention or an explicit goal vector.

We evaluate all models on in-distribution objects and unseen goals over three seeds, and report results in Table II. BiREPO achieves the highest average success rate across the three primitives and the highest pushing success. BiREPO-ACT achieves the second-highest average success across the primitives. The adapted DP3 baseline is less competitive, particularly for pushing and rotation. Qualitatively, its rollouts capture the coarse motion but are less precise during contact approach and termination, resulting in occasional toppling or target overshoot. Figure 4 shows that the overall trends for rotation persist across broad sweeps of the position and orientation thresholds that determine the success criteria.

The ablations show that the goal vector and cross-attention contribute differently across primitives. For pushing, the displaced goal cloud already conveys the desired translation, so removing the goal vector has little effect. For rotation, where the current and goal clouds share position and differ



Fig. 5: Objects used in our real-world experiments. The top row shows five regular box-shaped objects, and the bottom row shows five irregular (nonconvex) objects with varied geometry.

mainly in orientation, removing the goal vector reduces success from (70.63% to 59.92%). Cross-attention provides a consistent benefit: removing it lowers success to 69.2%, 64.09%, and 70.38% for pushing, rotating, and flipping, respectively. Flipping remains robust across BiREPO variants, because success mainly requires reaching and settling on the target face, without requiring precise planar positioning.

C. Real-world Setup and Results

We evaluate BiREPO on a real bimanual platform similar to Zhao et al. [1], using two arms with closed parallel-jaw grippers. Two RealSense D415 cameras provide fused point clouds, which we crop to just the workspace. Fig. 5 shows the objects, and Fig. 6 shows example rollouts. Unless otherwise noted, we use the simulation criteria (Sec. IV-B) for evaluation. Pushing and rotating require final xy position and orientation errors below 30 mm and 10° respectively. Flipping requires rotation to within 10° of the desired 90° and settling on the target face after the arms withdraw. Because ground-truth object poses are unavailable on hardware, we measure the final pose error by Iterative Closest Point(ICP) registration of the observed final point cloud to the goal point cloud P^{goal} , initialized at the commanded goal transform.

BiREPO Zero-shot Deployment. We deploy the simulation-trained policies zero-shot on unseen real-world objects. As shown in Table III, BiREPO achieves 65%,

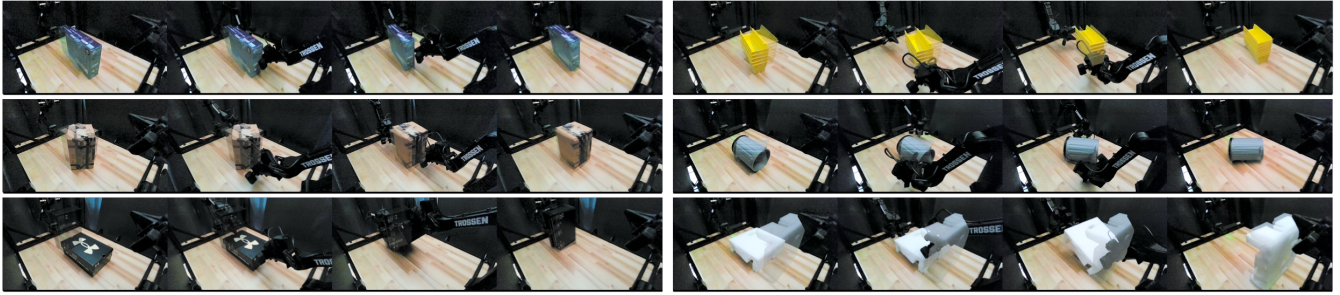


Fig. 6: Representative real-world successful executions of bimanual pushing (top), rotating (middle), and flipping (bottom) using BiREPO. Each row shows a regular object on the left and an irregular object on the right, with each sequence progressing from left to right. An overlay on the image indicates the desired object pose.

Task	Regular Objects		Irregular Objects		All
	Easy	Hard	Easy	Hard	
Pushing	20 / 25	16 / 25	16 / 25	13 / 25	65 / 100
Rotating	17 / 25	16 / 25	16 / 25	14 / 25	63 / 100
Flipping	22 / 25	14 / 25	18 / 25	16 / 25	70 / 100

TABLE III: Zero-shot real-world success rates of BiREPO on regular and irregular objects. Hard trials request pushes beyond 10 cm, rotations beyond 45° , or flips whose support edge is oblique to the world frame axes, requiring the arms to twist while tipping; easy trials are the complementary cases. Each entry reports successful trials out of 25, with 100 trials per primitive.

	BiNoMaP	BiREPO (Ours)
Training Data	1 Human Video	Simulation Rollouts
Sim2Real Transfer	Not needed	Needed
Goal Specification	Predefined primitive	Target poses
Execution	Open-loop	Replan from observations
Per-object Tuning	≤ 10 trials	None
White Foam	Pushing: 7 / 10	Pushing: 8 / 10
Trash Can	Poking: 5 / 10	Flipping: 9 / 10
Brown Box	Pivoting: 0 / 10	Flipping: 6 / 10

TABLE IV: Comparison with BiNoMaP [33]. The upper rows summarize design differences between the two approaches. The lower rows report our implementation of both methods on the same hardware for three approximately matched object pose changes, with 10 trials each. Because the methods use different primitive definitions and goal parameterizations, these results are not intended as a controlled one-to-one comparison (see Sec. V-C for discussion).

63%, and 70% success for pushing, rotating, and flipping, respectively. Average success decreases by about 9% relative to simulation, likely due to limited variation in physical parameters during training. For flipping, slippery surfaces can cause contact slip or prevent the object from settling.

Baseline Comparison. We additionally compare with BiNoMaP [33], a state-of-the-art approach to bimanual nonprehensile manipulation. BiNoMaP learns category-level task-space primitives from human video and geometric optimization, whereas BiREPO learns goal-conditioned policies from simulation-generated demonstrations. Table IV shows these differences and reports our implementations on the same hardware for three approximately matched object pose changes. We use pushing for both, while BiNoMaP’s pivoting and poking primitives are the closest primitives to our selected flipping goals. Because BiNoMaP specifies goals

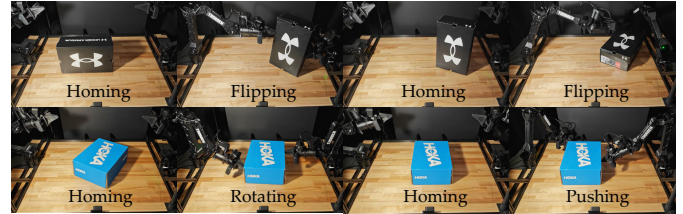


Fig. 7: Multi-step composition of BiREPO primitives. Top: two consecutive flips of the black shoebox. Bottom: a rotation followed by a push of the blue box.

qualitatively (e.g., “flip the trash can upright”) instead of as a target pose, Table IV scores flipping at the task level for both methods. A trial succeeds if the object comes to rest on the intended face with no position or orientation tolerance. This matches BiNoMaP’s evaluation and the intent of our flipping primitive. For pushing we instead keep the pose-based criterion used in Table III, since both methods reach 10/10 under the relaxed criterion; our pushing evaluation also samples diverse goal directions and scores orientation, which makes it a stricter test. The primitive definitions and goal parameterizations differ, so these results are not a controlled comparison. Instead, they highlight tradeoffs: BiNoMaP avoids simulation and uses limited human supervision, while BiREPO avoids human demonstrations, conditions on target object poses, and replans from observations.

Since BiNoMaP is not publicly available yet, we implement it to the best of our ability and use its human video and hand-tracking pipeline. In our setup, hand tracking became less reliable under occlusion or when hands left the camera view, producing inaccurate trajectories. Some trajectories were also infeasible because the corresponding end-effector motions approached or exceeded the arms’ reachable workspace, particularly for larger pose changes. These observations are specific to our implementation and experimental setup.

Chaining Primitives. In our dataset, each primitive begins and ends at a shared robot home pose, enabling repeated execution and composition across primitives. Figure 7 shows two consecutive flips of the black shoebox, and a separate sequence consisting of a rotation followed by a push of the blue box. Chaining is constrained by the workspace jointly reachable by both arms, which limits the set of primitive sequences that can be executed near workspace boundaries.

VI. LIMITATIONS

BiREPO has several limitations. First, while BiREPO replans from new observations, it lacks an explicit recovery mechanism, limiting correction after large execution deviations. Second, the diversity of learned behaviors remains constrained by the designed primitives and contact generation procedure. Expanding and composing primitives could support a broader pose changes. Third, our simulation emphasizes geometric diversity with limited variation in physical parameters such as friction and mass, which can reduce sim-to-real robustness. Future work could broaden physical variation to improve transfer to the real world.

VII. CONCLUSION

We present BiREPO, a framework for learning goal-conditioned bimanual nonprehensile manipulation across diverse object geometries. BiREPO combines wrench-guided contact projection with simulation-based demonstration generation to train policies for pushing, rotating, and flipping. Experiments in simulation and on a real dual-arm platform demonstrate these capabilities on regular and irregular objects. We hope that this work inspires future research on scalable, long-horizon bimanual nonprehensile manipulation.

REFERENCES

- [1] T. Z. Zhao *et al.*, “Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware,” in *Robotics: Science and Systems (RSS)*, 2023.
- [2] F. Krebs *et al.*, “A Bimanual Manipulation Taxonomy,” in *IEEE Robotics and Automation Letters (RA-L)*, 2022.
- [3] M. T. Mason, “Mechanics and Planning of Manipulator Pushing Operations,” in *International Journal of Robotics Research*, 1986.
- [4] —, “Progress in Nonprehensile Manipulation,” in *International Journal of Robotics Research (IJRR)*, 1999.
- [5] C. Chi *et al.*, “Diffusion Policy: Visuomotor Policy Learning via Action Diffusion,” in *Robotics: Science and Systems (RSS)*, 2023.
- [6] Y. Ze *et al.*, “3D Diffusion Policy: Generalizable Visuomotor Policy Learning via Simple 3D Representations,” in *Robotics: Science and Systems (RSS)*, 2024.
- [7] NVIDIA *et al.*, “Isaac Lab: A GPU-Accelerated Simulation Framework for Multi-Modal Robot Learning,” *arXiv preprint arXiv:2511.04831*, 2025.
- [8] J. Grannen *et al.*, “Stabilize to Act: Learning to Coordinate for Bimanual Manipulation,” in *Conference on Robot Learning (CoRL)*, 2023.
- [9] I.-C. A. Liu *et al.*, “VoxAct-B: Voxel-Based Acting and Stabilizing Policy for Bimanual Manipulation,” in *Conference on Robot Learning (CoRL)*, 2024.
- [10] Y. Zhong *et al.*, “A Survey on Vision-Language-Action Models: An Action Tokenization Perspective,” *arXiv preprint arXiv:2507.01925*, 2025.
- [11] P. Intelligence *et al.*, “ $\pi_{0.7}$: a Steerable Generalist Robotic Foundation Model with Emergent Capabilities,” *arXiv preprint arXiv:2604.15483*, 2026.
- [12] —, “ $\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization,” in *Conference on Robot Learning (CoRL)*, 2025.
- [13] V. Lim *et al.*, “Planar Robot Casting with Real2Sim2Real Self-Supervised Learning,” in *International Conference on Robotics and Automation (ICRA)*, 2022.
- [14] Z. Jiang *et al.*, “DexMimicGen: Automated Data Generation for Bimanual Dexterous Manipulation via Imitation Learning,” in *International Conference on Robotics and Automation (ICRA)*, 2025.
- [15] A. Mandekar *et al.*, “MimicGen: A Data Generation System for Scalable Robot Learning using Human Demonstrations,” in *Conference on Robot Learning (CoRL)*, 2023.
- [16] M. Moghani *et al.*, “SoftMimicGen: A Data Generation System for Scalable Robot Learning in Deformable Object Manipulation,” in *International Conference on Robotics and Automation (ICRA)*, 2026.
- [17] S. Deng *et al.*, “GraspVLA: a Grasping Foundation Model Pre-trained on Billion-scale Synthetic Action Data,” in *Conference on Robot Learning (CoRL)*, 2025.
- [18] R. Wang *et al.*, “DexGraspNet: A large-scale robotic dexterous grasp dataset for general objects based on simulation,” in *International Conference on Robotics and Automation (ICRA)*, 2023.
- [19] J. Zhang *et al.*, “DexGraspNet 2.0: Learning Generative Dexterous Grasping in Large-scale Synthetic Cluttered Scenes,” in *Conference on Robot Learning (CoRL)*, 2024.
- [20] J. He *et al.*, “DexVLG: Dexterous Vision-Language-Grasp Model at Scale,” in *International Conference on Computer Vision (ICCV)*, 2025.
- [21] Y. Li *et al.*, “Learning Geometry-Aware Nonprehensile Pushing and Pulling with Dexterous Hands,” in *International Conference on Robotics and Automation (ICRA)*, 2026.
- [22] Y. Wang *et al.*, “Learning Dual-Arm Coordination for Grasping Large Flat Objects,” in *International Conference on Robotics and Automation (ICRA)*, 2025.
- [23] N. Chavan-Dafle *et al.*, “Extrinsic dexterity: In-hand manipulation with external forces,” in *International Conference on Robotics and Automation (ICRA)*, 2014.
- [24] W. Yang *et al.*, “Dynamic On-Palm Manipulation via Controlled Sliding,” in *Robotics: Science and Systems (RSS)*, 2024.
- [25] S. Chen *et al.*, “Synthesizing Dexterous Nonprehensile Pregrasp for Ungraspable Objects,” in *ACM SIGGRAPH*, 2023.
- [26] Y. Cho *et al.*, “CORN: Contact-based Object Representation for Nonprehensile Manipulation of General Unseen Objects,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [27] J. Lyu *et al.*, “DyWA: Dynamics-adaptive World Action Model for Generalizable Non-prehensile Manipulation,” in *International Conference on Computer Vision (ICCV)*, 2025.
- [28] W. Zhou *et al.*, “HACMan: Learning Hybrid Actor-Critic Maps for 6D Non-Prehensile Manipulation,” in *Conference on Robot Learning (CoRL)*, 2023.
- [29] —, “Learning to Grasp the Ungraspable with Emergent Extrinsic Dexterity,” in *Conference on Robot Learning (CoRL)*, 2022.
- [30] W. C. Agboh *et al.*, “Multi-object grasping in the plane,” in *International Symposium on Robotics Research (ISRR)*, 2022.
- [31] A. Zeng *et al.*, “Learning Synergies between Pushing and Grasping with Self-supervised Deep Reinforcement Learning,” in *International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [32] Y. Wang *et al.*, “Learning Dual-Arm Push and Grasp Synergy in Dense Clutter,” in *IEEE Robotics and Automation Letters (RA-L)*, 2025.
- [33] H. Zhou *et al.*, “BiNoMaP: Learning Category-Level Bimanual Non-Prehensile Manipulation Primitives,” *arXiv preprint arXiv:2509.21256*, 2025.
- [34] X. Li *et al.*, “Planning-Guided Diffusion Policy Learning for Generalizable Contact-Rich Bimanual Manipulation,” in *International Conference on Robotics and Automation (ICRA)*, 2026.
- [35] L. Yang *et al.*, “Physics-Driven Data Generation for Contact-Rich Manipulation via Trajectory Optimization,” in *Robotics: Science and Systems (RSS)*, 2025.
- [36] L. Downs *et al.*, “Google Scanned Objects: A High-Quality Dataset of 3D Scanned Household Items,” in *International Conference on Robotics and Automation (ICRA)*, 2022.
- [37] B. Sundaralingam *et al.*, “cuRobo: Parallelized Collision-Free Minimum-Jerk Robot Motion Generation,” *arXiv preprint arXiv:2310.17274*, 2023.
- [38] Y. Zhou *et al.*, “On the Continuity of Rotation Representations in Neural Networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [39] W. Peebles *et al.*, “Scalable Diffusion Models with Transformers,” in *International Conference on Computer Vision (ICCV)*, 2023.
- [40] J. Ho *et al.*, “Denoising Diffusion Probabilistic Models,” in *Neural Information Processing Systems (NeurIPS)*, 2020.
- [41] A. Q. Nichol *et al.*, “Improved Denoising Diffusion Probabilistic Models,” in *International Conference on Machine Learning (ICML)*, 2021.
- [42] J. Song *et al.*, “Denoising Diffusion Implicit Models,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [43] K. Sohn *et al.*, “Learning Structured Output Representation Using Deep Conditional Generative Models,” in *Neural Information Processing Systems (NeurIPS)*, 2015.
- [44] O. Ronneberger *et al.*, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *MICCAI*, 2015.